

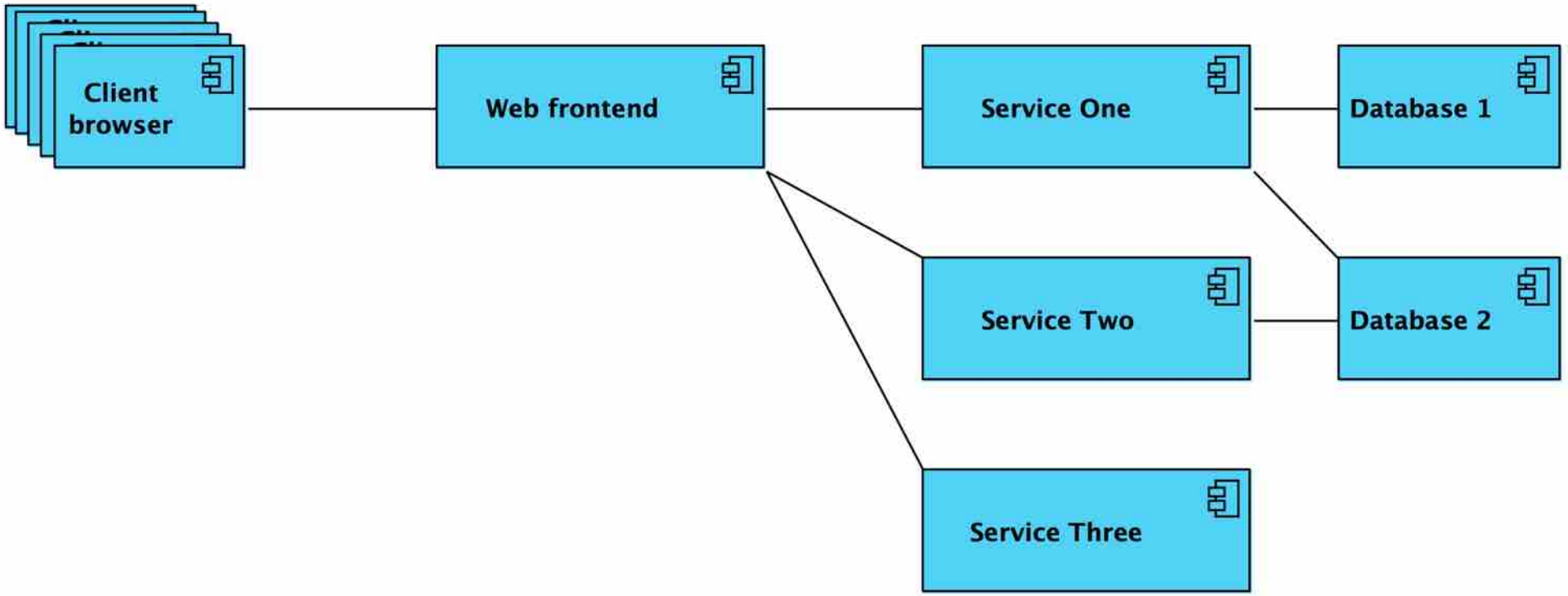
ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ
Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών
Τεχνολογίες Υπηρεσιών Λογισμικού

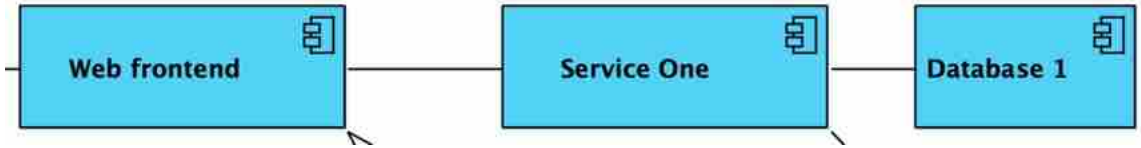
Τεχνολογίες Υπηρεσιών Λογισμικού

Β.Βεσκούκης, Καθ.ΕΜΠ (v.vescoukis@cs.ntua.gr)

API Gateways

API-based backend






```html
<label>
  User ID:
  <input type="text" id="userId" value="007">
</label>
<br>
<label>
  User Name:
  <input type="text" id="userName" value="James">
</label>
<br><br>

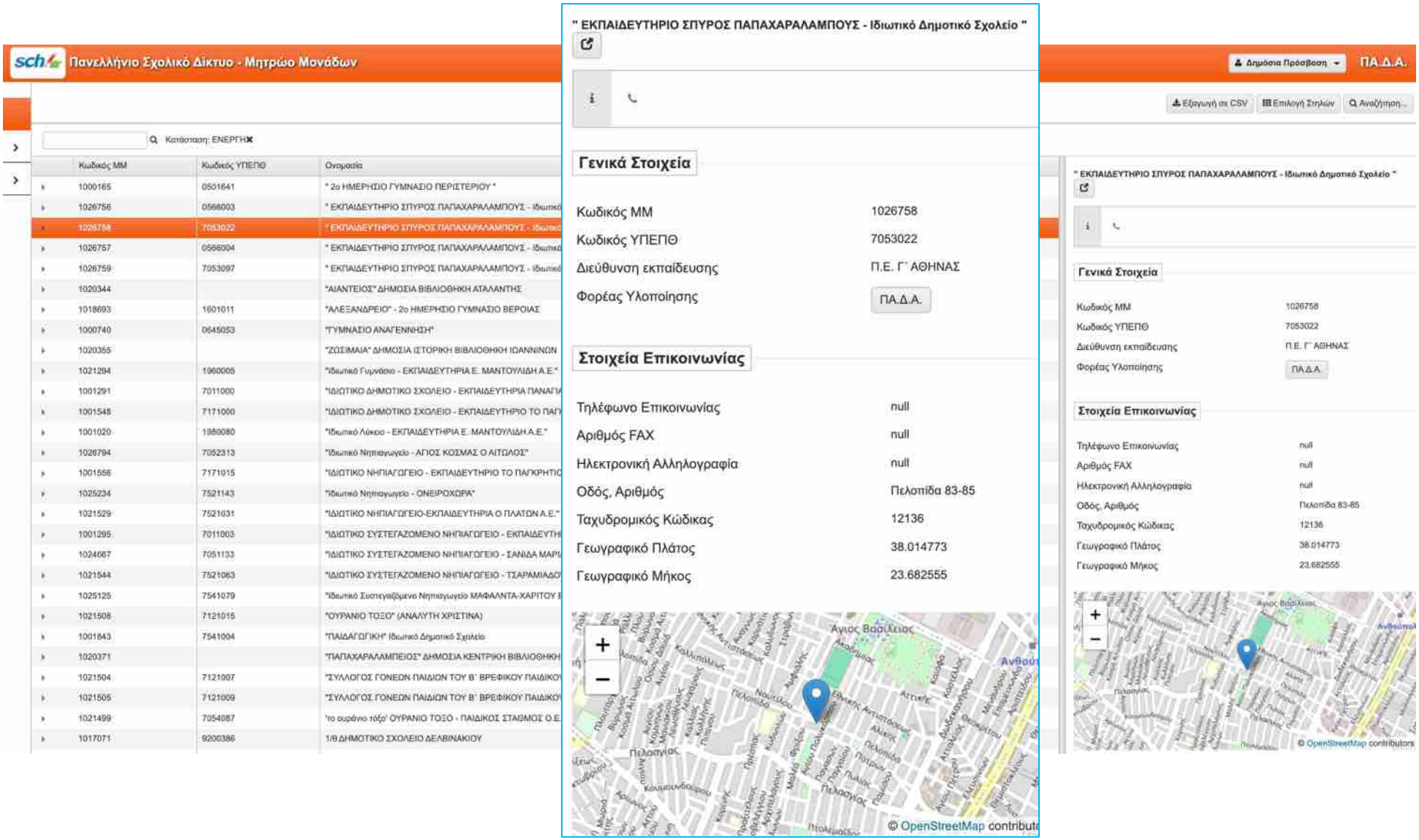
<button onclick="callServiceOne()">Call Service One</button>
<button onclick="callServiceTwo()">Call Service Two</button>
<button onclick="callServiceThree()">Call Service Three</button>

<div id="output"></div>
```

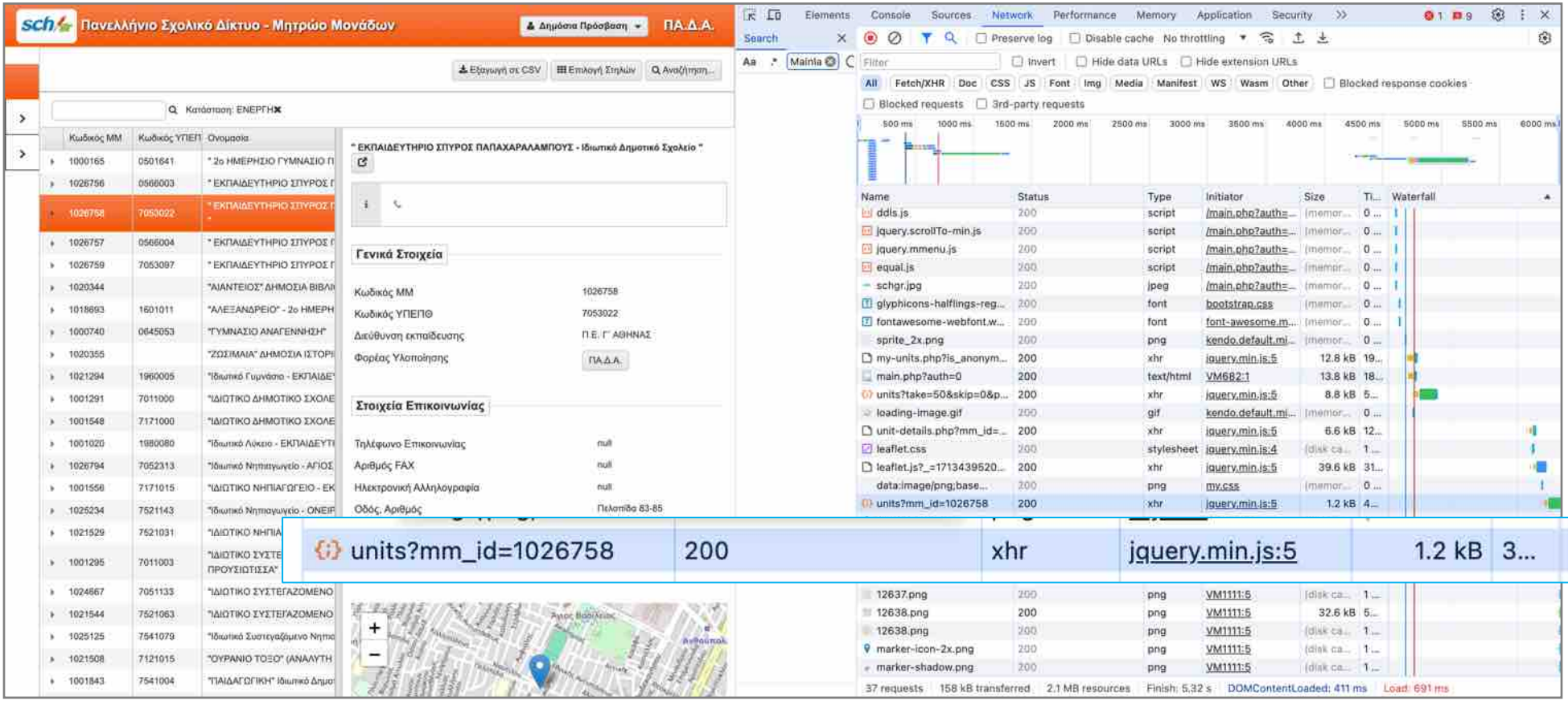
```javascript
async function callServiceOne() {
  const userId = document.getElementById('userId').value;
  const userName = document.getElementById('userName').value;

  const params = new URLSearchParams({ id: userId, name: userName });
  const response = await fetch(`https://service-one.example.com/api?${params}`);
  const data = await response.text();
  document.getElementById('output').innerText = `Service One Response: ${data}`;
}
```

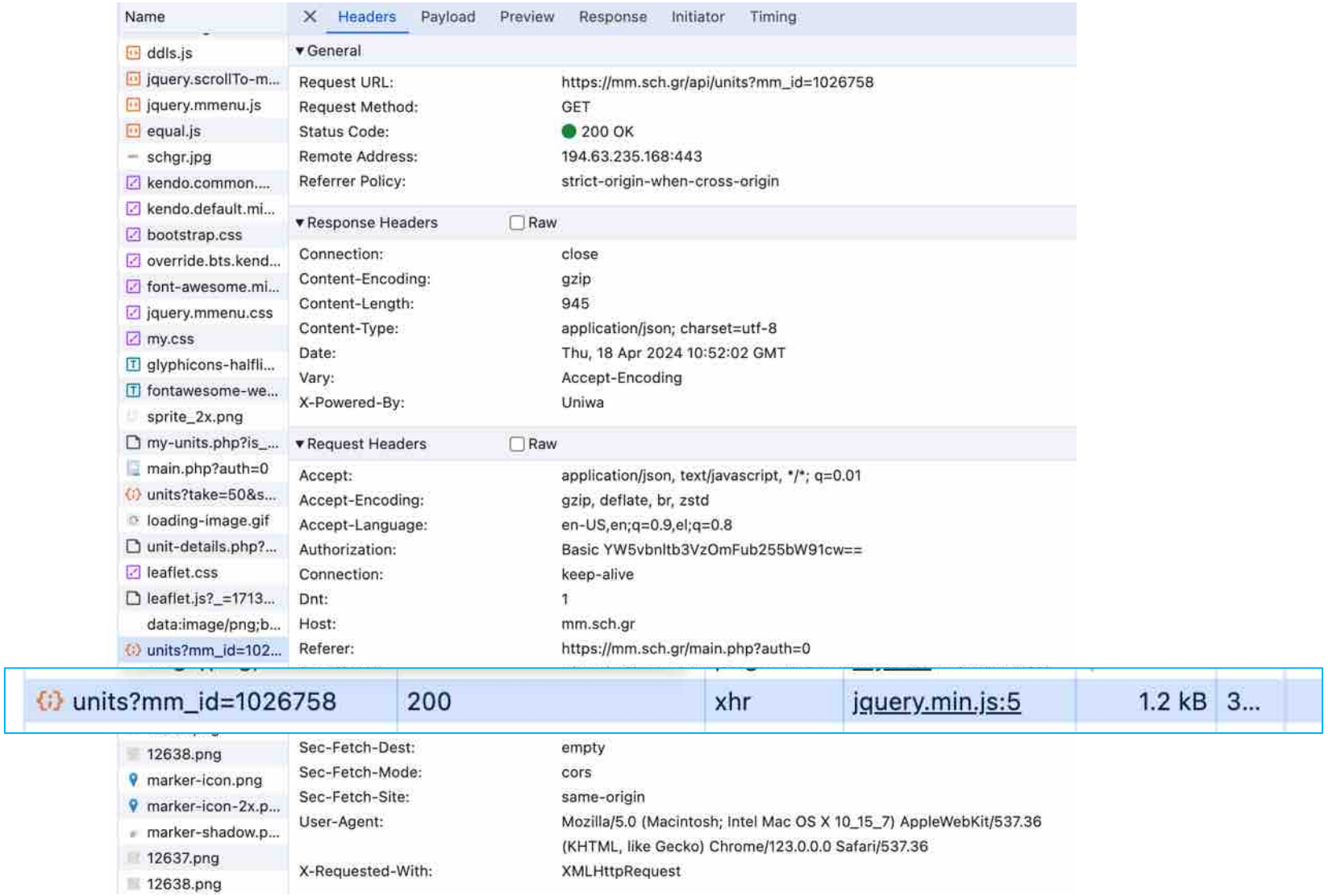
No API gateway = unwanted revelations



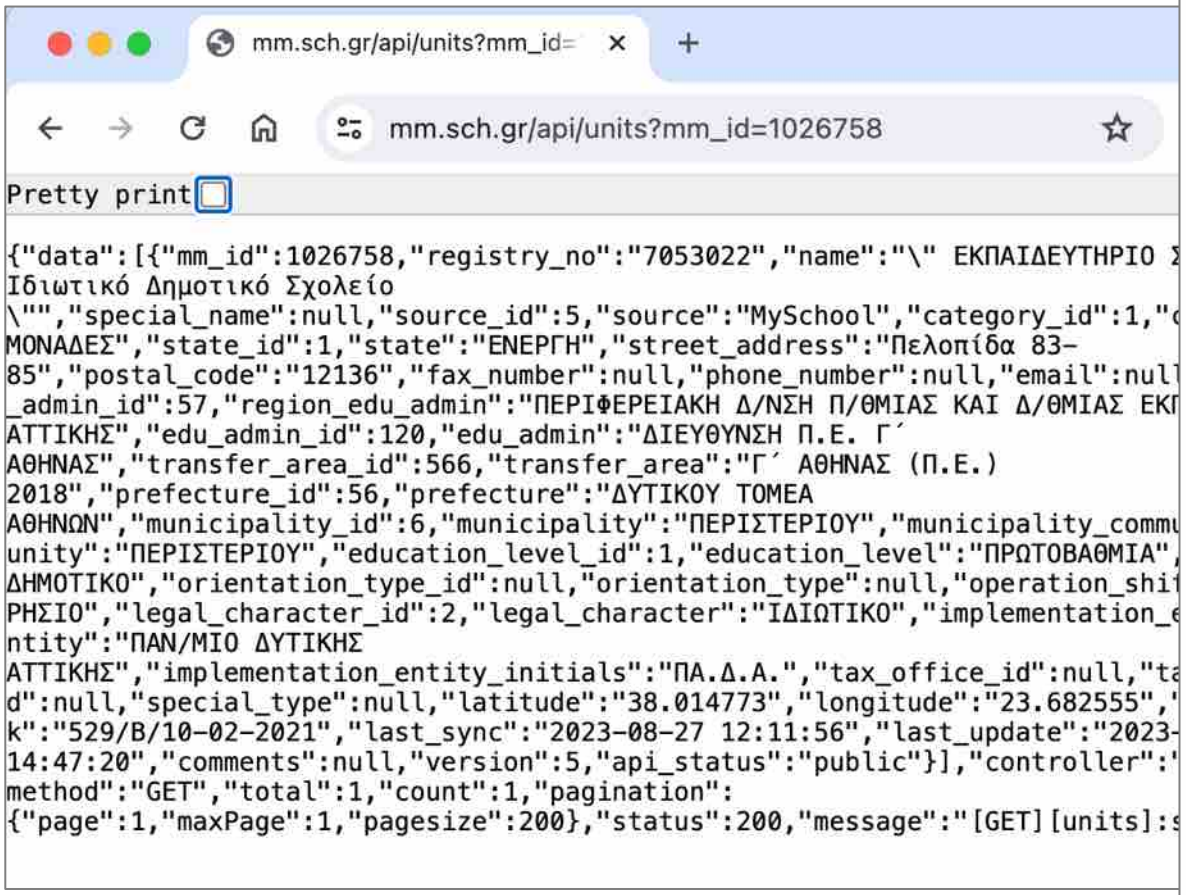
No API gateway = unwanted revelations



No API gateway = unwanted revelations



No API gateway





Management of requests to backend

Load balancing

- ...self-explanatory

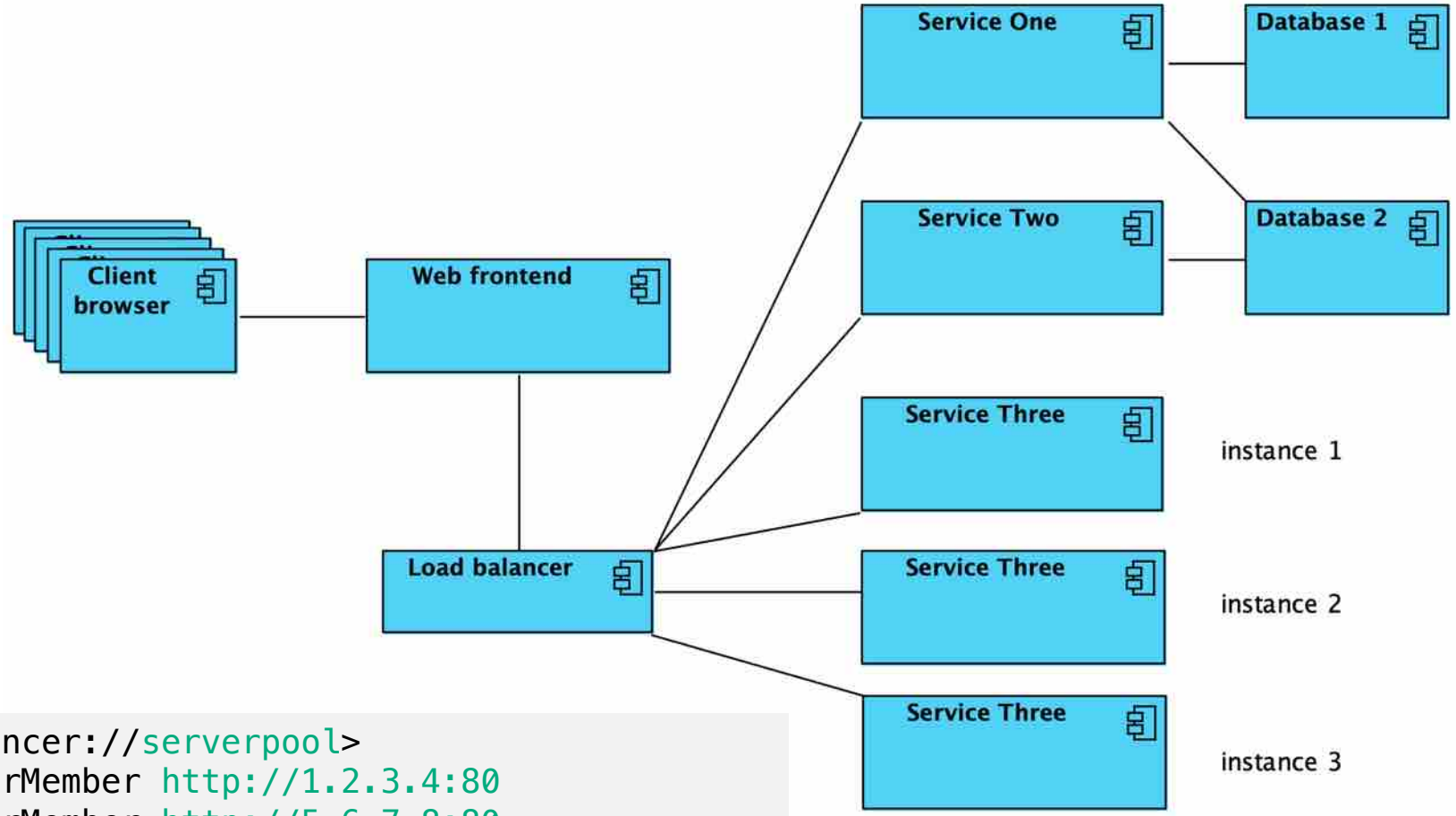
Reverse proxying

- Map ports/services to paths

API gateways

- Plus: quotas, auth, more

Load balancing



Apache

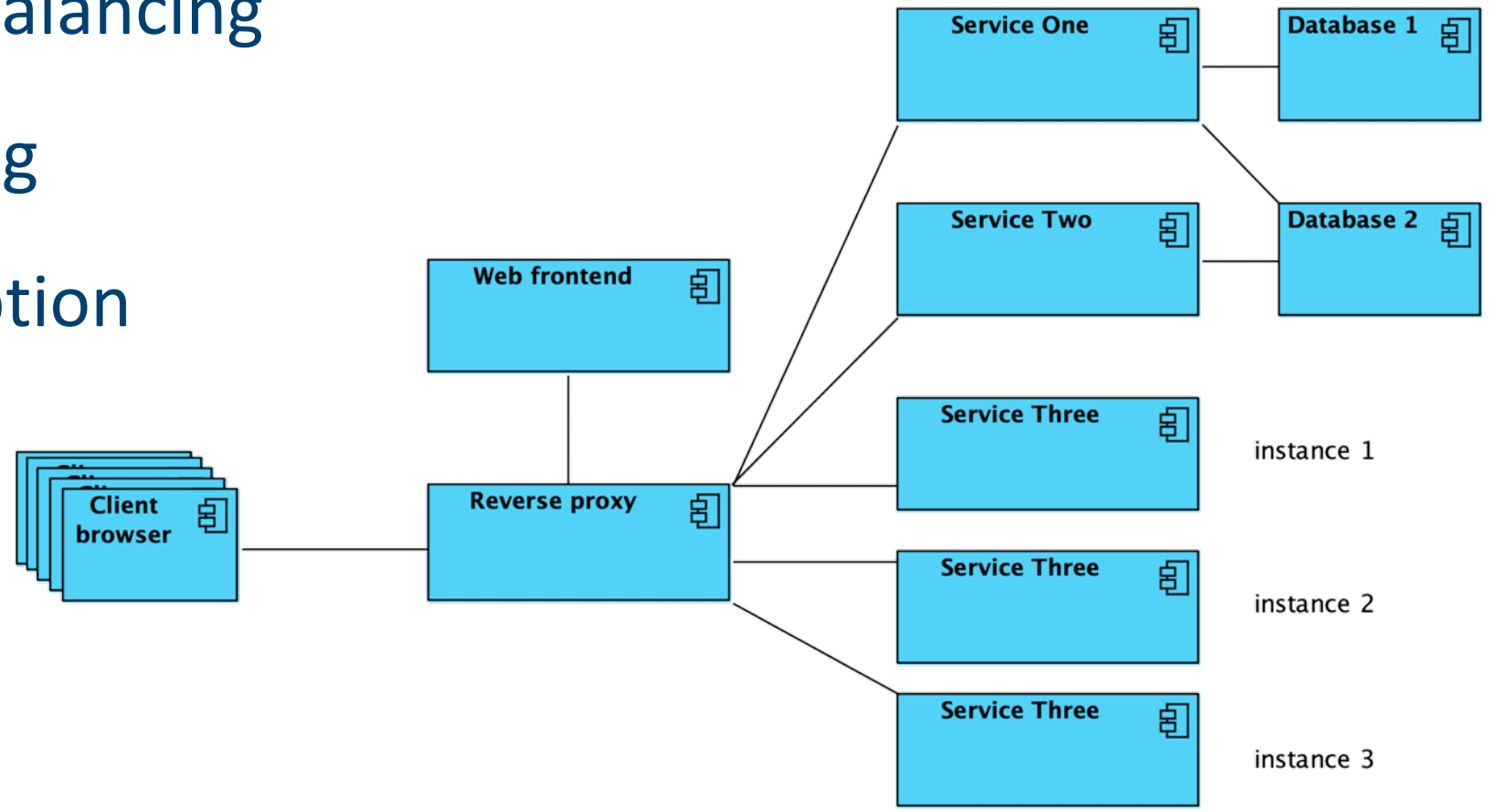
```
<proxy balancer://serverpool>
    BalancerMember http://1.2.3.4:80
    BalancerMember http://5.6.7.8:80
    ProxySet lbmethod=bytraffic
</proxy>
ProxyPass "/site" "balancer://serverpool/"
ProxyPassReverse "/site" "balancer://serverpool/"
```

Reverse proxy

Load balancing

Caching

Encryption



Balancing on Apache

```
<VirtualHost *:443>
    ServerName service-one.example.com

    ProxyRequests Off
    ProxyPreserveHost On

    <Proxy "balancer://serviceonecluster">
        BalancerMember http://10.0.0.101:6000
        BalancerMember http://10.0.0.102:6000
        BalancerMember http://10.0.0.103:6000
        ProxySet lbmethod=byrequests
    </Proxy>

    ProxyPass "/api" "balancer://serviceonecluster/api"
    ProxyPassReverse "/api" "balancer://serviceonecluster/api"

</VirtualHost>
```

Reverse proxy: ailog.softlab.ntua.gr

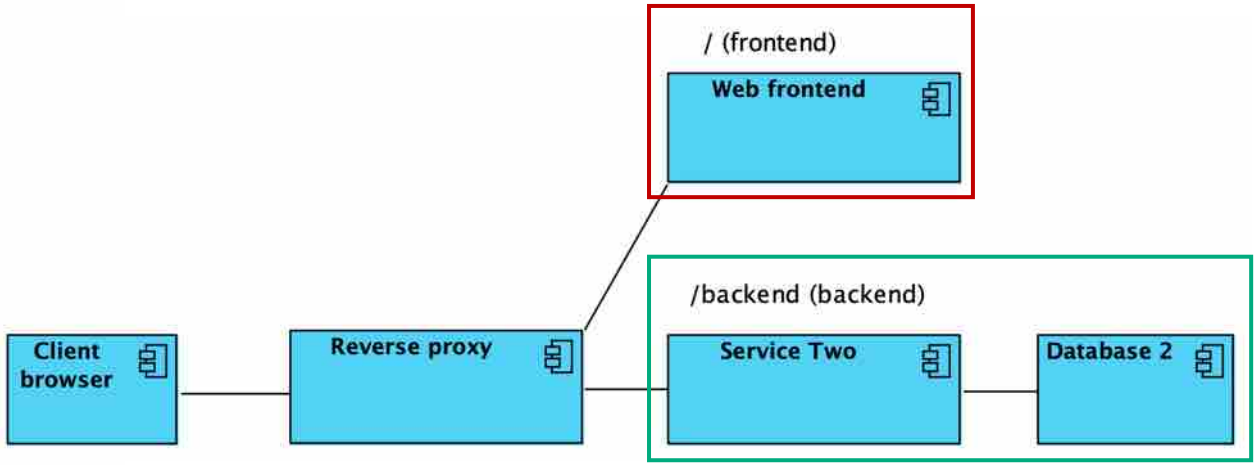
```
ServerAdmin webmaster@localhost
DocumentRoot /var/www/ailog.softlab.ntua.gr
ServerName ailog.softlab.ntua.gr
ServerAlias ailog.softlab.ece.ntua.gr
# ...
RewriteEngine On
ProxyPreserveHost On
```

```
<Directory /var/www/ailog.softlab.ntua.gr>
    Require ip 147.102.0.0/16
</Directory>

<Proxy *>
    Require ip 147.102.0.0/16
</Proxy>
```

```
ProxyPass /backend/ http://localhost:5000/
ProxyPassReverse /backend/ http://localhost:5000/
```

```
ProxyPass / http://localhost:3001/
ProxyPassReverse / http://localhost:3001/
```



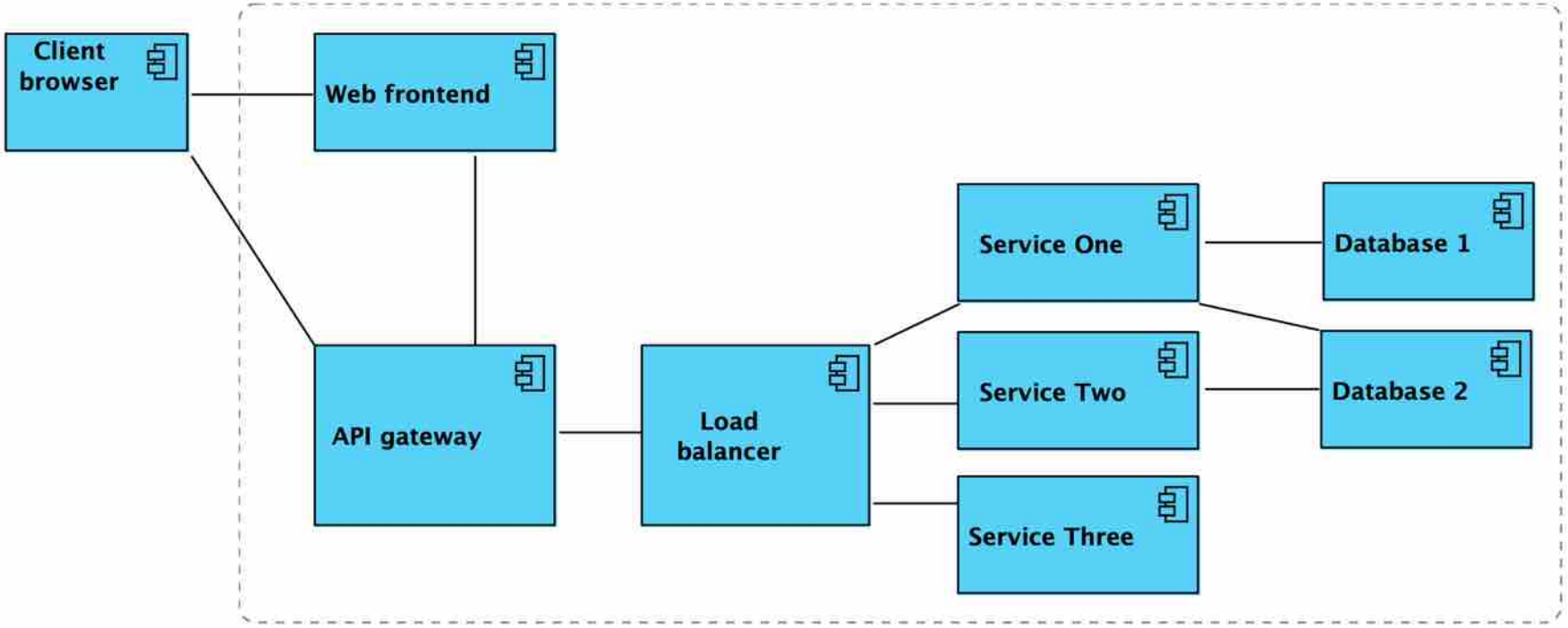
API gateway: "ReverseProxy+++"

API Gateway Services

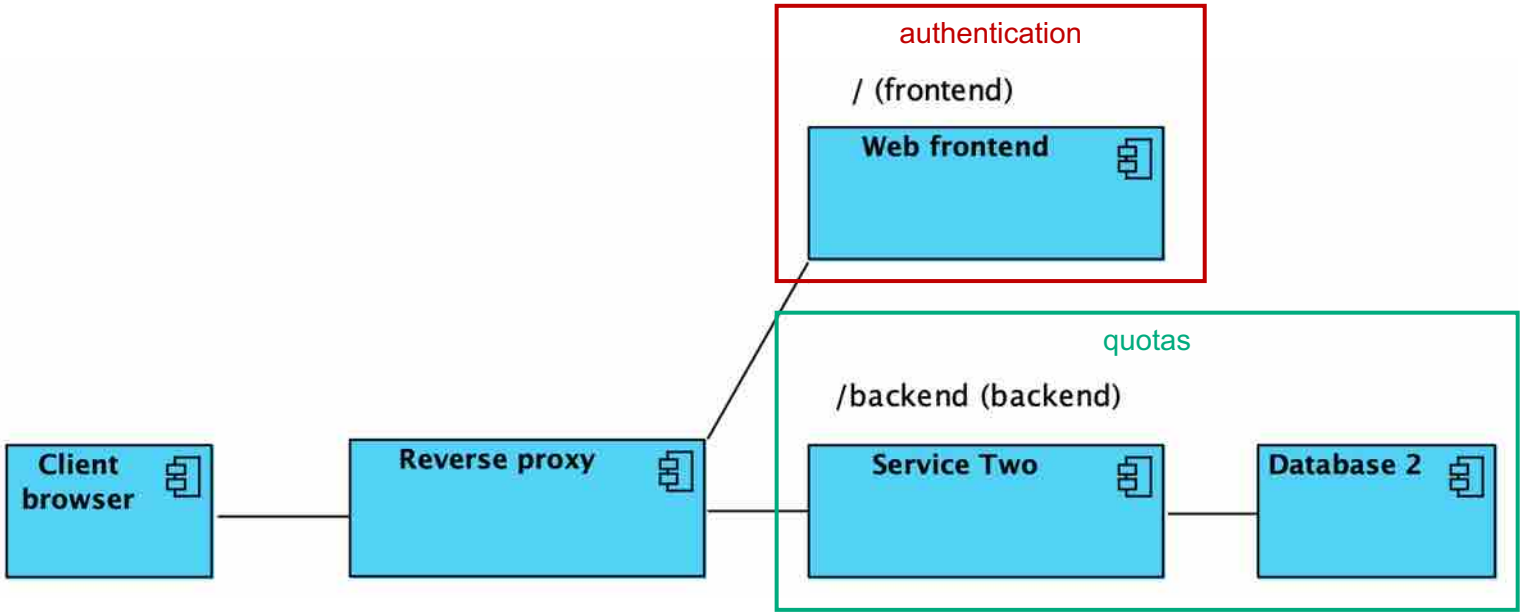
- Authentication
- Authorization
- IP whitelisting
- Quotas
- Logging
- Tracing
- More

Function	Load Balancer	Reverse Proxy	API Gateway
Balance incoming requests	✅	✅	✅
URL Rewrite		✅	✅
Caching		✅	✅
Prevention from Attack			✅
Protocol Translation			✅
Authentication			✅
IP Whitelisting			✅
Rate Limiting, Quota			✅
Logging			✅
Tracing			✅
Health Checking			✅
Dynamic Config Updates			✅

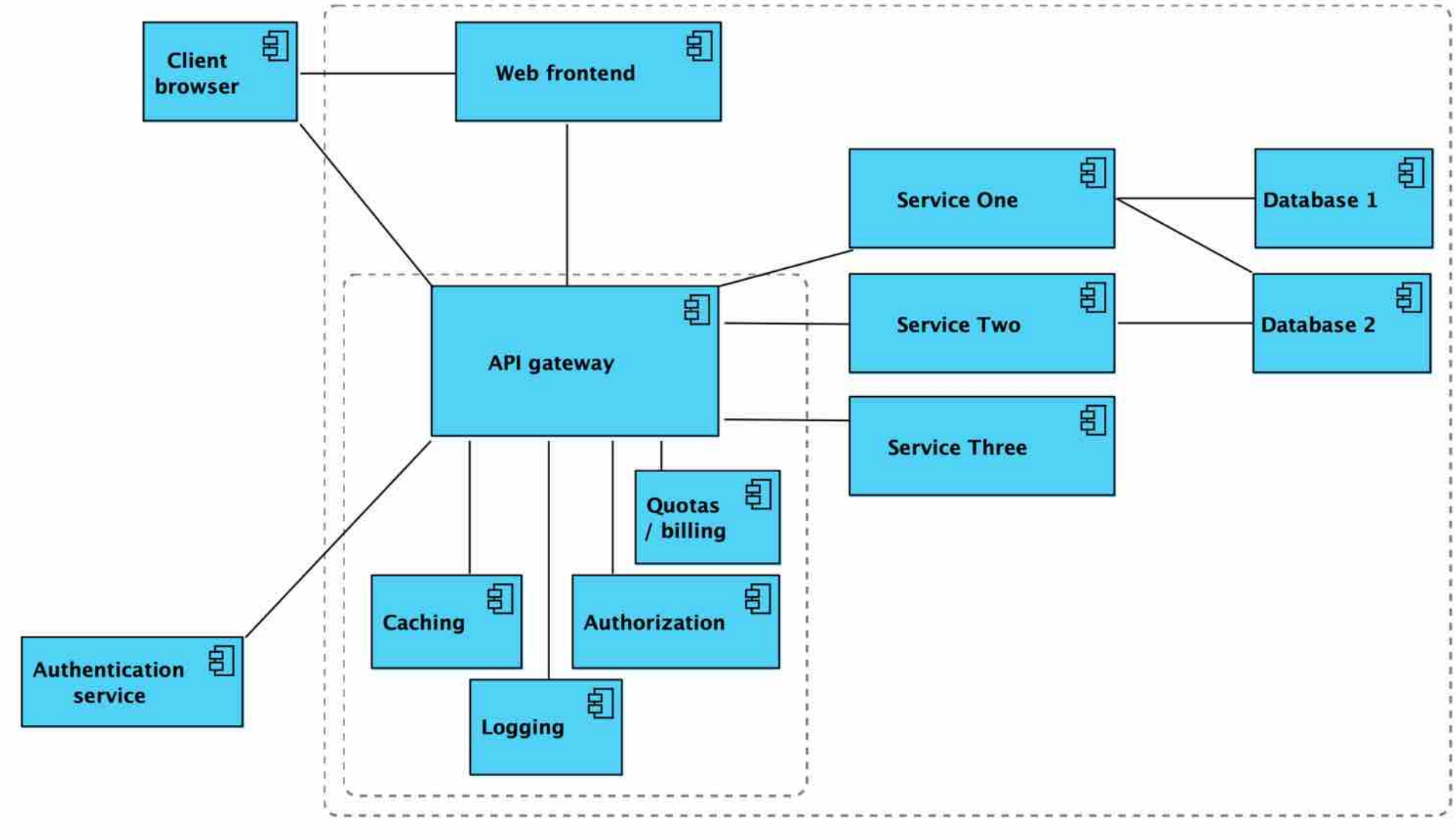
API gateway


Client browser
Web frontend
API gateway
Load balancer
Service One
Service Two
Service Three
Database 1
Database 2

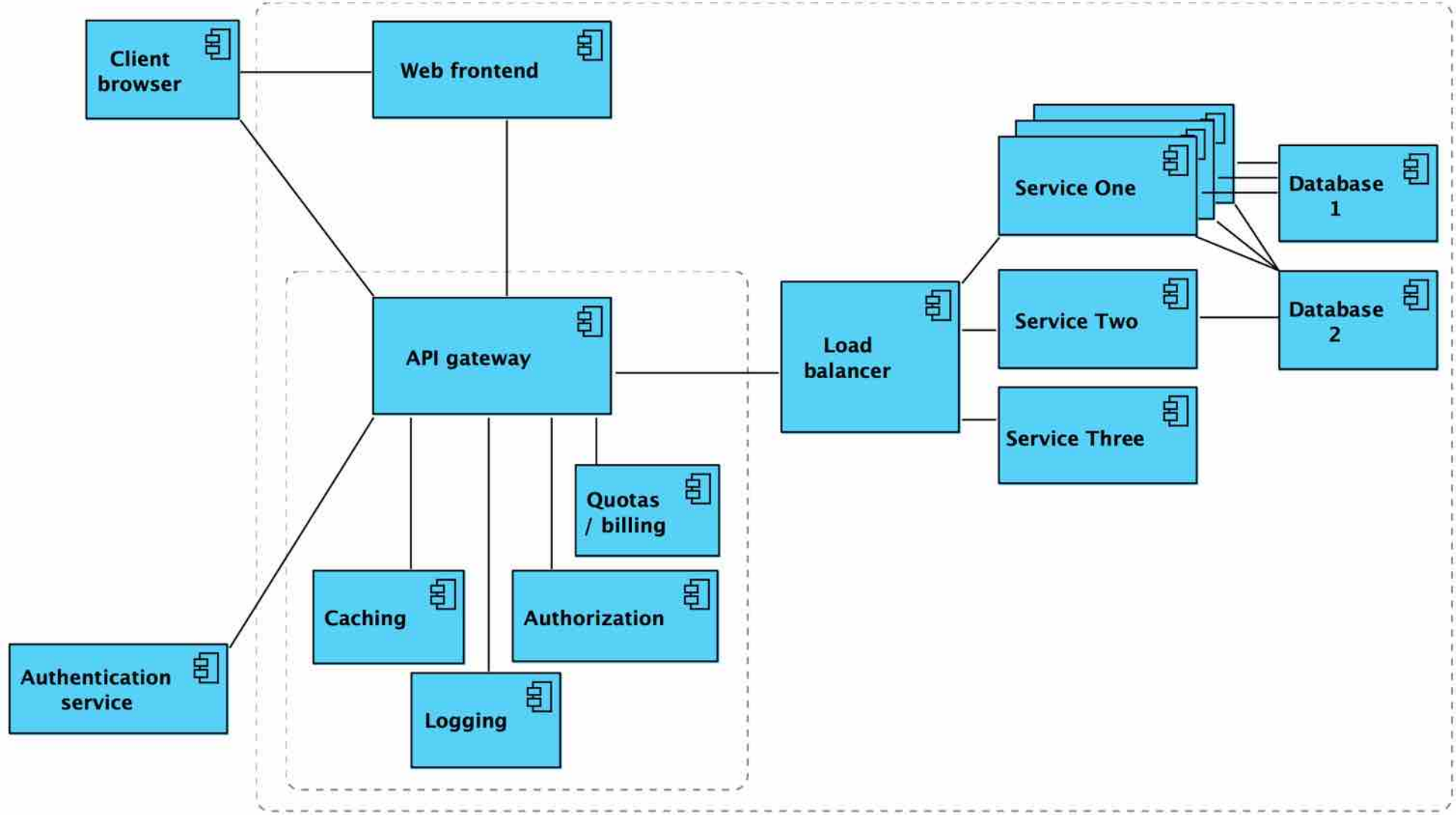
API gateways



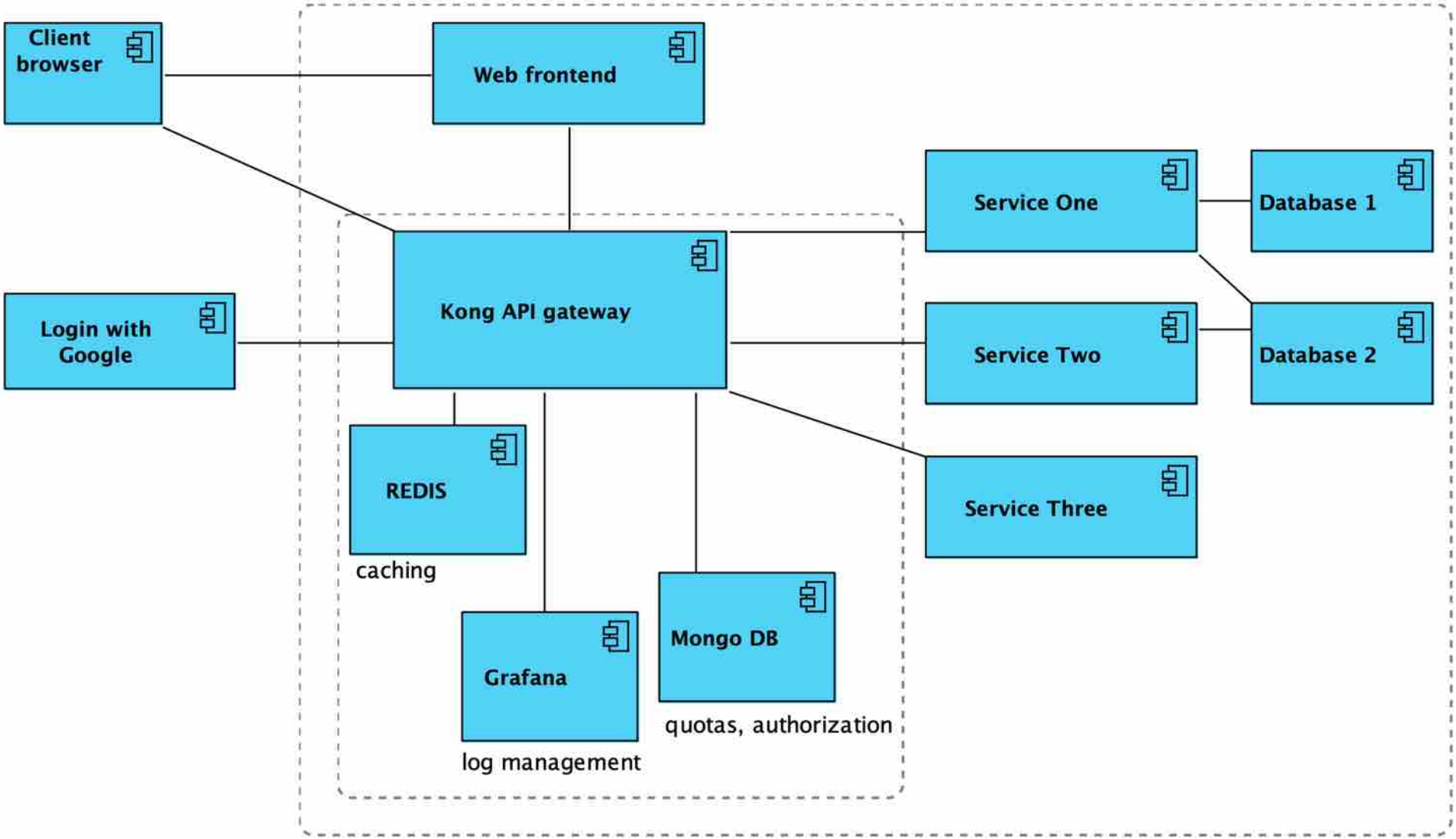
API gateways



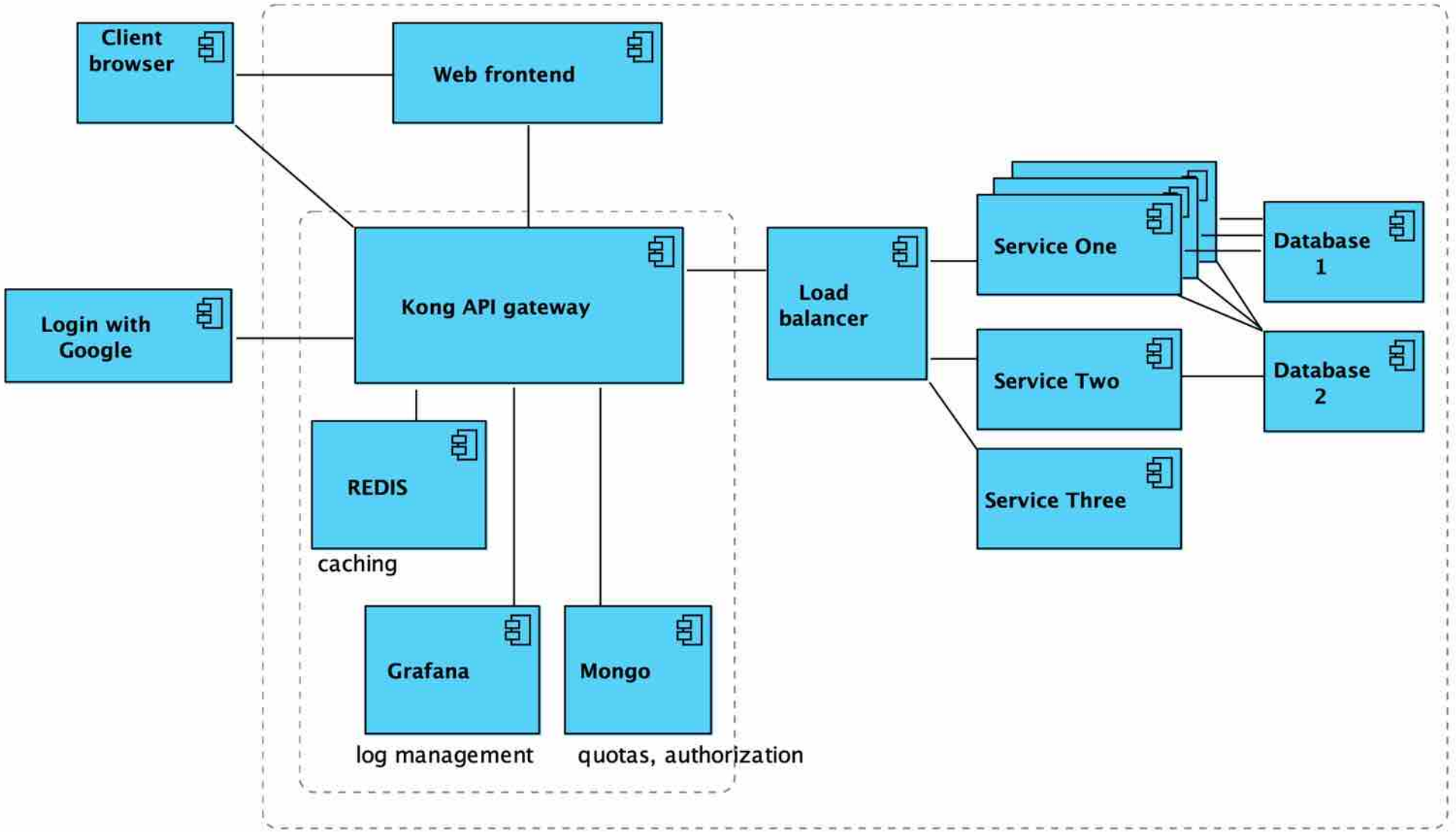
API gateways



API gateways



API gateways - Kong



API gateways - Kong

```
$ sudo apt install kong
...
$ curl -i -X POST http://localhost:8001/services/ \
  --data name=service-one \
  --data url=http://service-one.example.com
...
$ curl -i -X POST
http://localhost:8001/services/service-one/routes \
  --data paths[]="/api"
...
$ curl -i -X POST
http://localhost:8001/services/service-one/plugins \
  --data name=rate-limiting \
  --data config.minute=1000 \
  --data config.policy=local
```

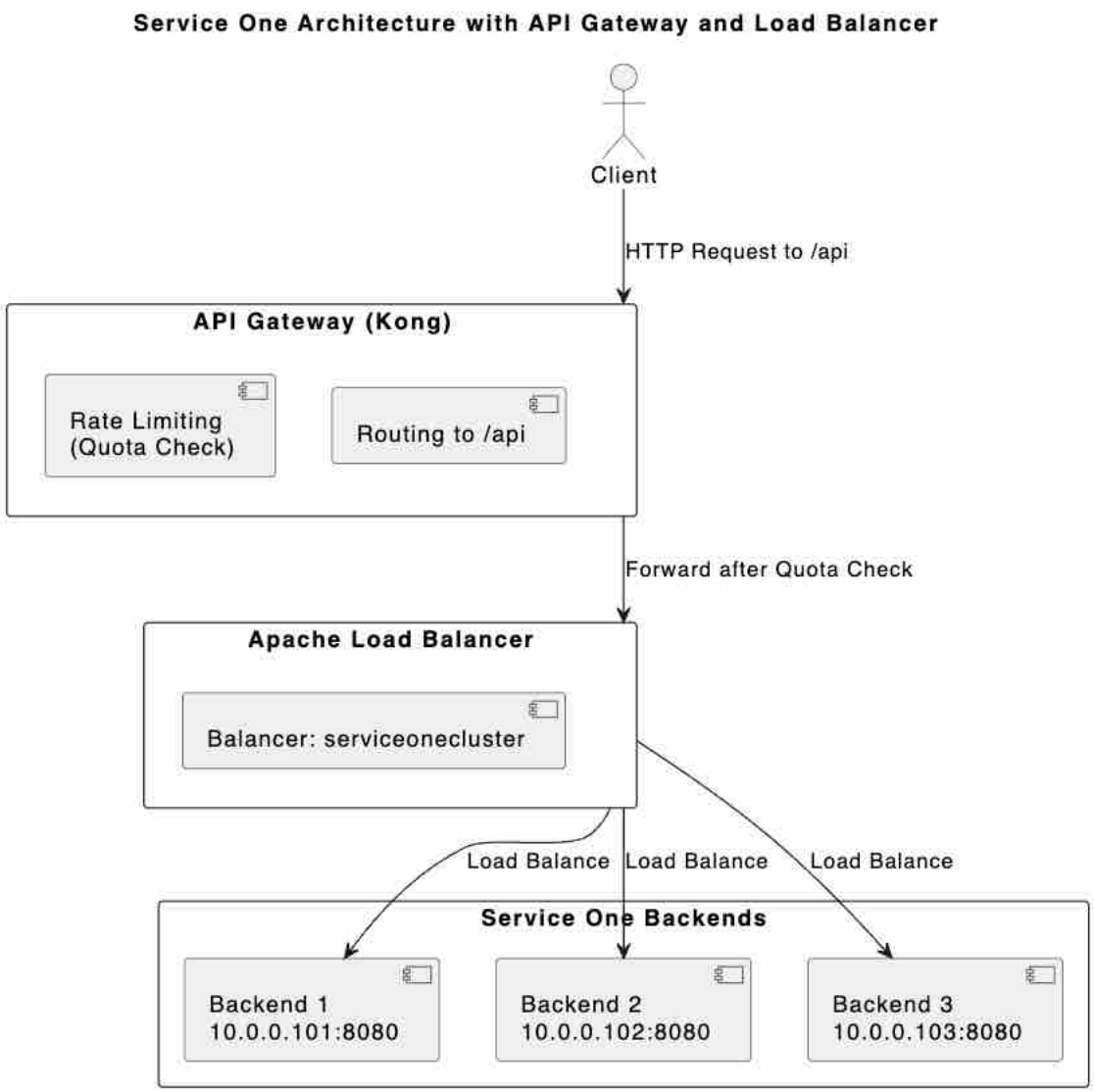
```
# kong.yaml

_format_version: "3.0"

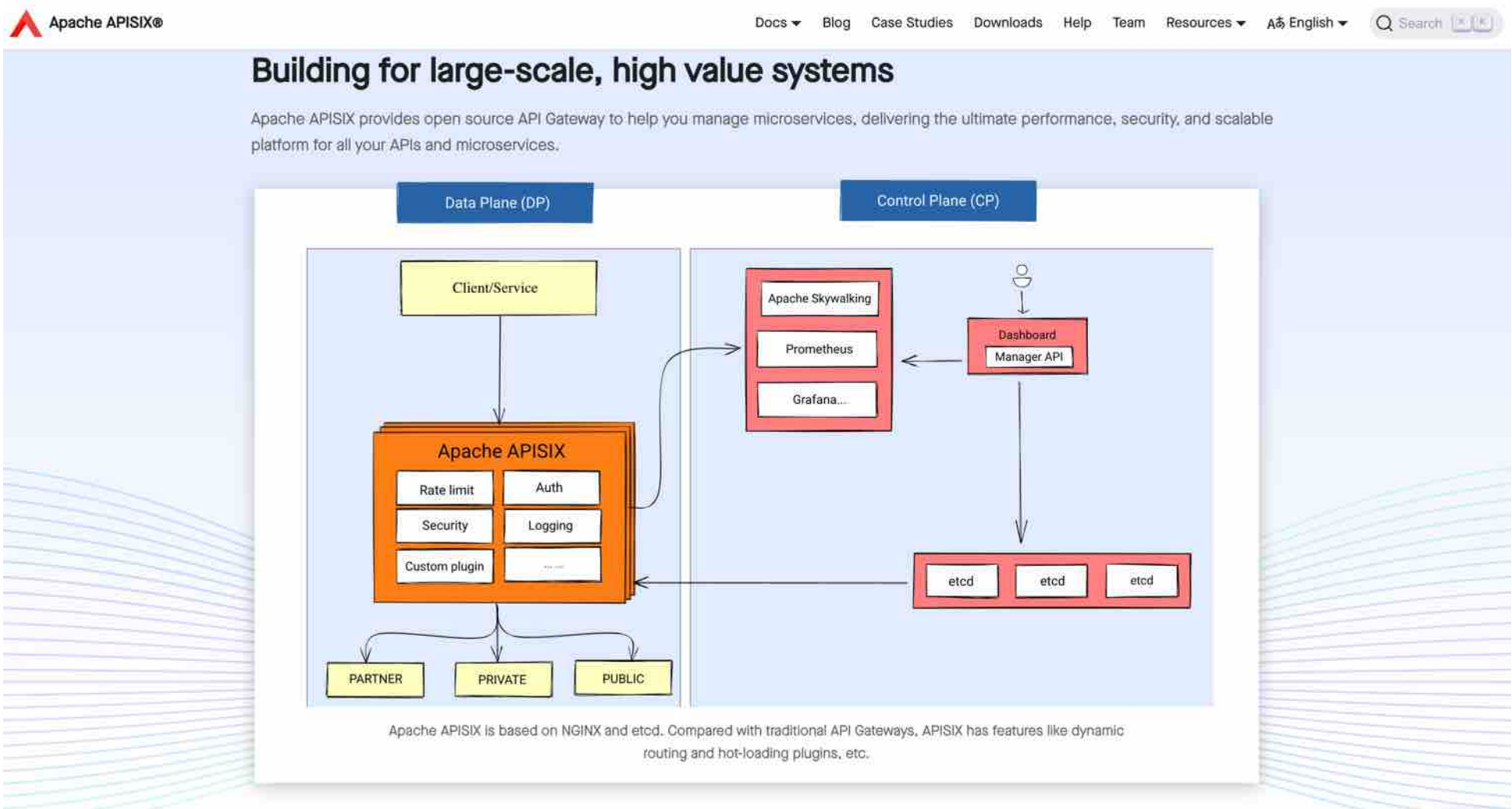
services:
  - name: service-one
    url: http://service-one.example.com
    routes:
      - name: service-one-api
        paths:
          - /api
        strip_path: false
    plugins:
      - name: rate-limiting
        config:
          minute: 1000   # Allow 1000 requests per minute
          policy: local  # Local in-memory counter

plugins:
  - name: rate-limiting
    service: service-one
    config:
      minute: 1000
      policy: local
```

API gateways - Kong



API gateways – Apache APISIX



API gateways – Apache APISIX

```
$ curl http://127.0.0.1:9080/apisix/admin/upstreams/1
-X PUT -d '
{
  "type": "roundrobin",
  "nodes": {
    "service-one.example.com:80": 1
  }
}'

...

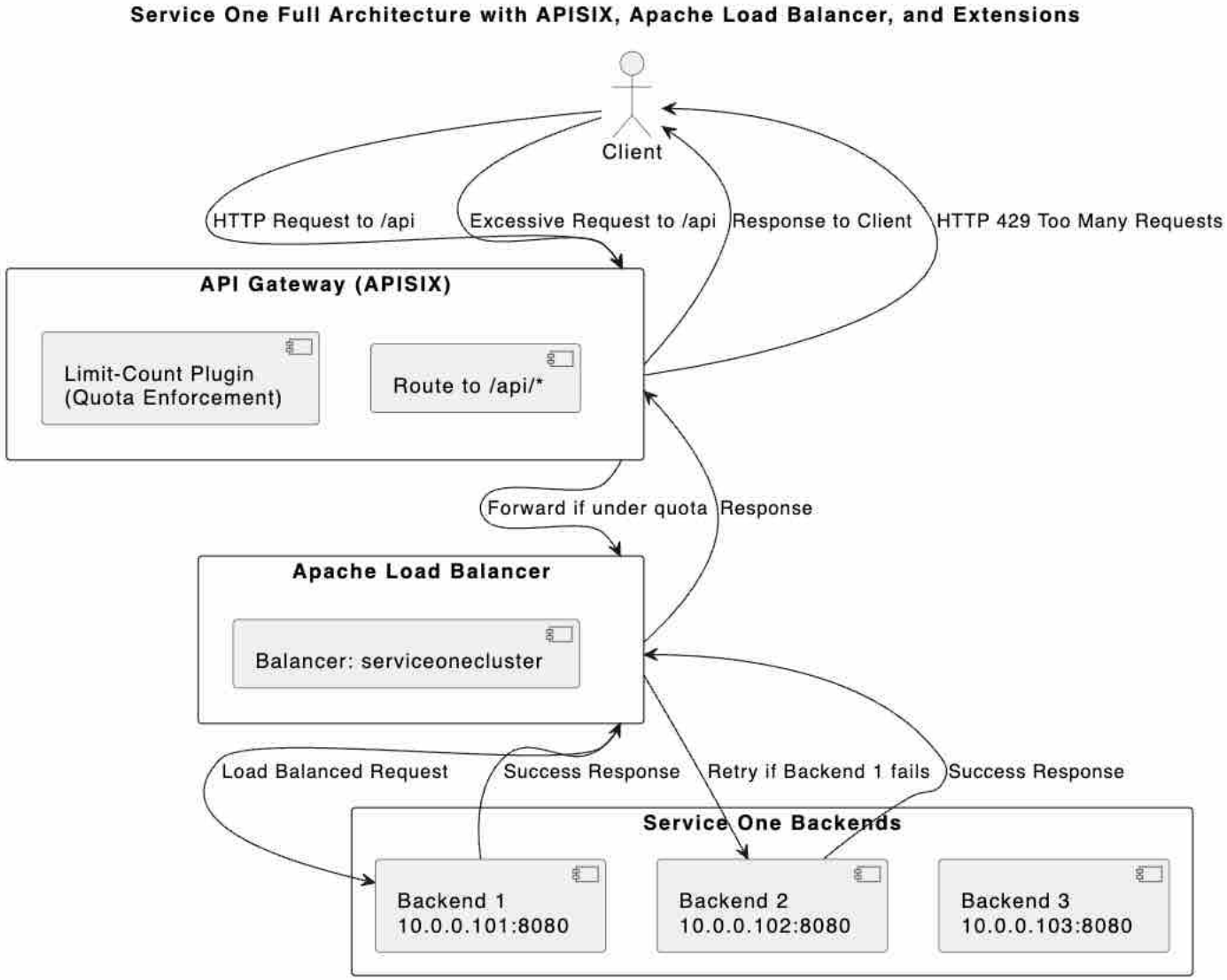
$ curl http://127.0.0.1:9080/apisix/admin/routes/1 -X
PUT -d '
{
  "uri": "/api/*",
  "upstream_id": 1,
  "plugins": {
    "limit-count": {
      "count": 1000,
      "time_window": 60,
      "key": "remote_addr",
      "rejected_code": 429,
      "policy": "local"
    }
  }
}'
```

```
# apisix.yaml

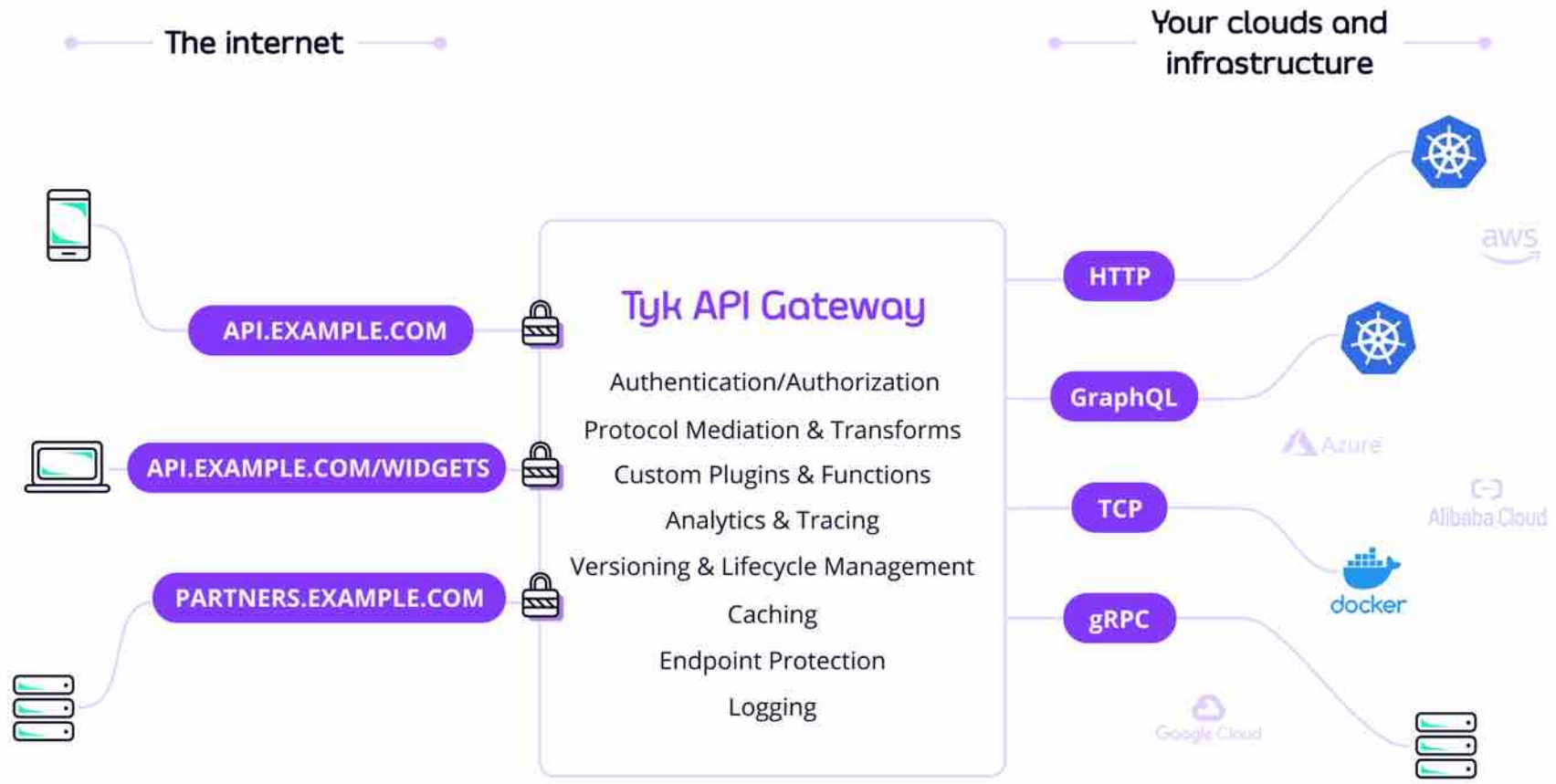
routes:
  - uri: /api/*
    name: service-one-api
    upstream_id: 1
    plugins:
      limit-count:
        count: 1000
        time_window: 60
        key: remote_addr
        rejected_code: 429
        policy: local

upstreams:
  - id: 1
    type: roundrobin
    nodes:
      "service-one.example.com:80": 1
```

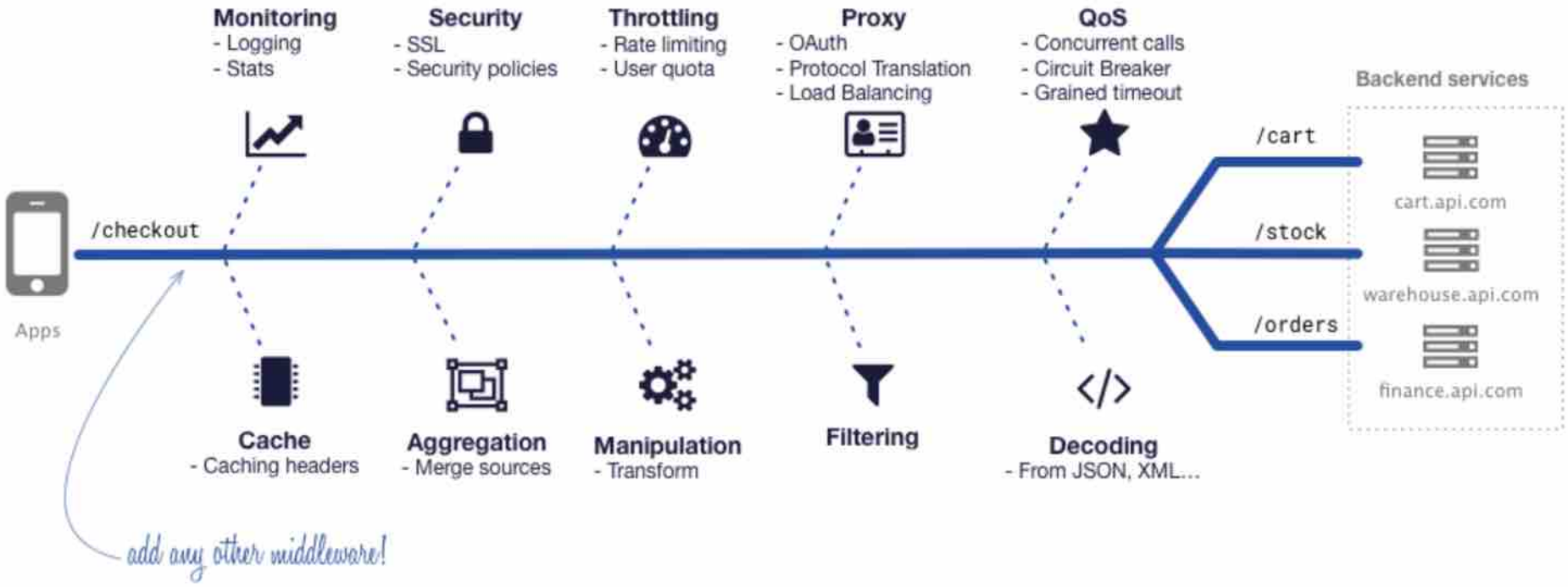
API gateways – Apache APISIX



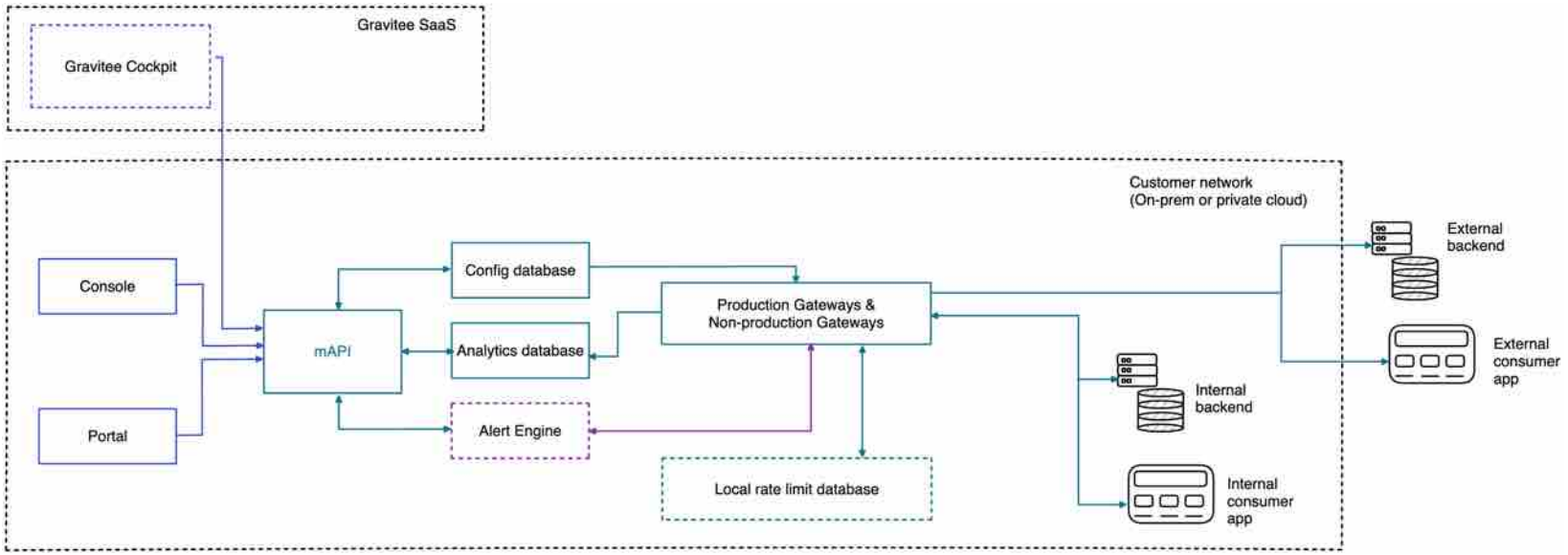
API gateways – Tyk



API gateways – Tyk



Gravitee open source

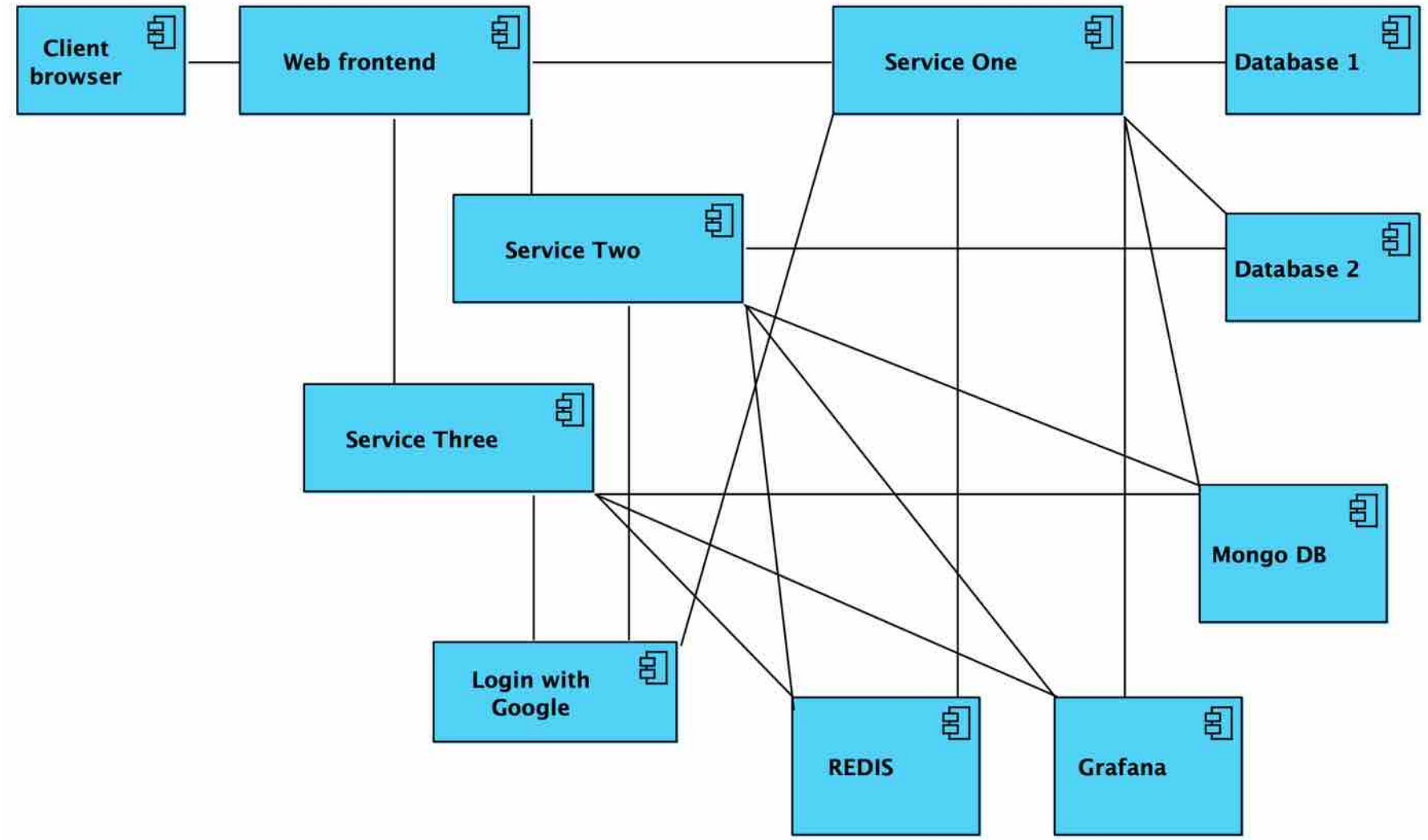


API gateways and microservices

Advantages

- Isolate microservice clients from microservices
- Mimimize impact on clients when updating microservice implementations
- Allow each microservice in many applications

To API-gateway or not to API-gateway?



Resources

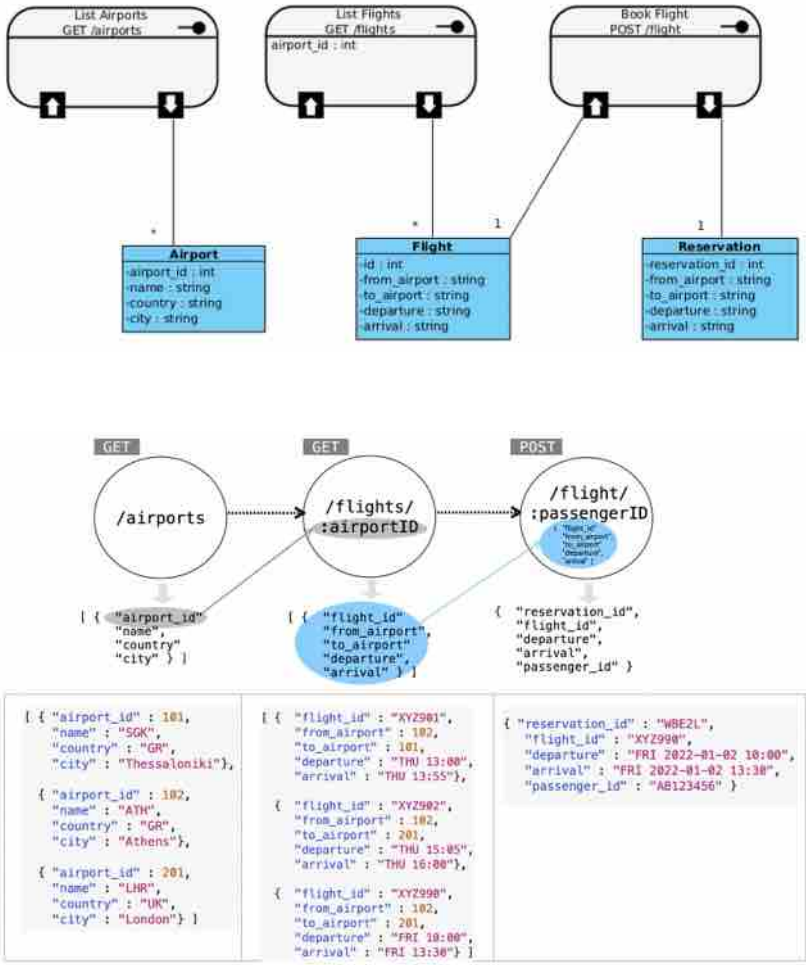
www.ibm.com/products/api-connect/api-gateway

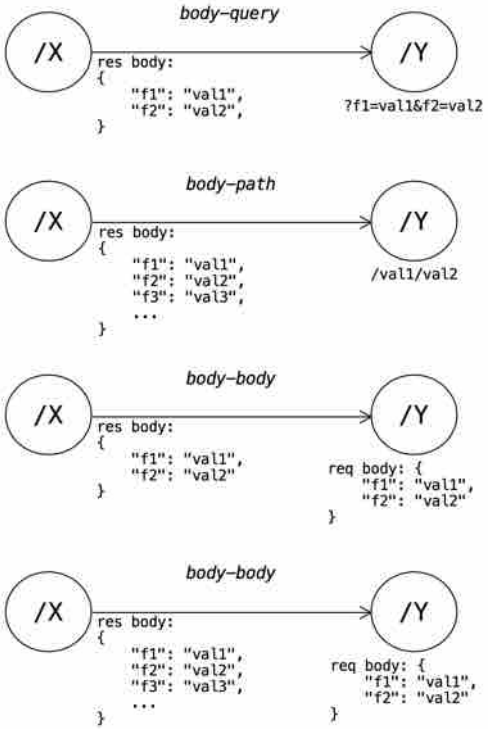
nordicapis.com/6-open-source-api-gateways/

dev.to/adisakshya/setting-up-an-api-gateway-for-your-microservices-3gal

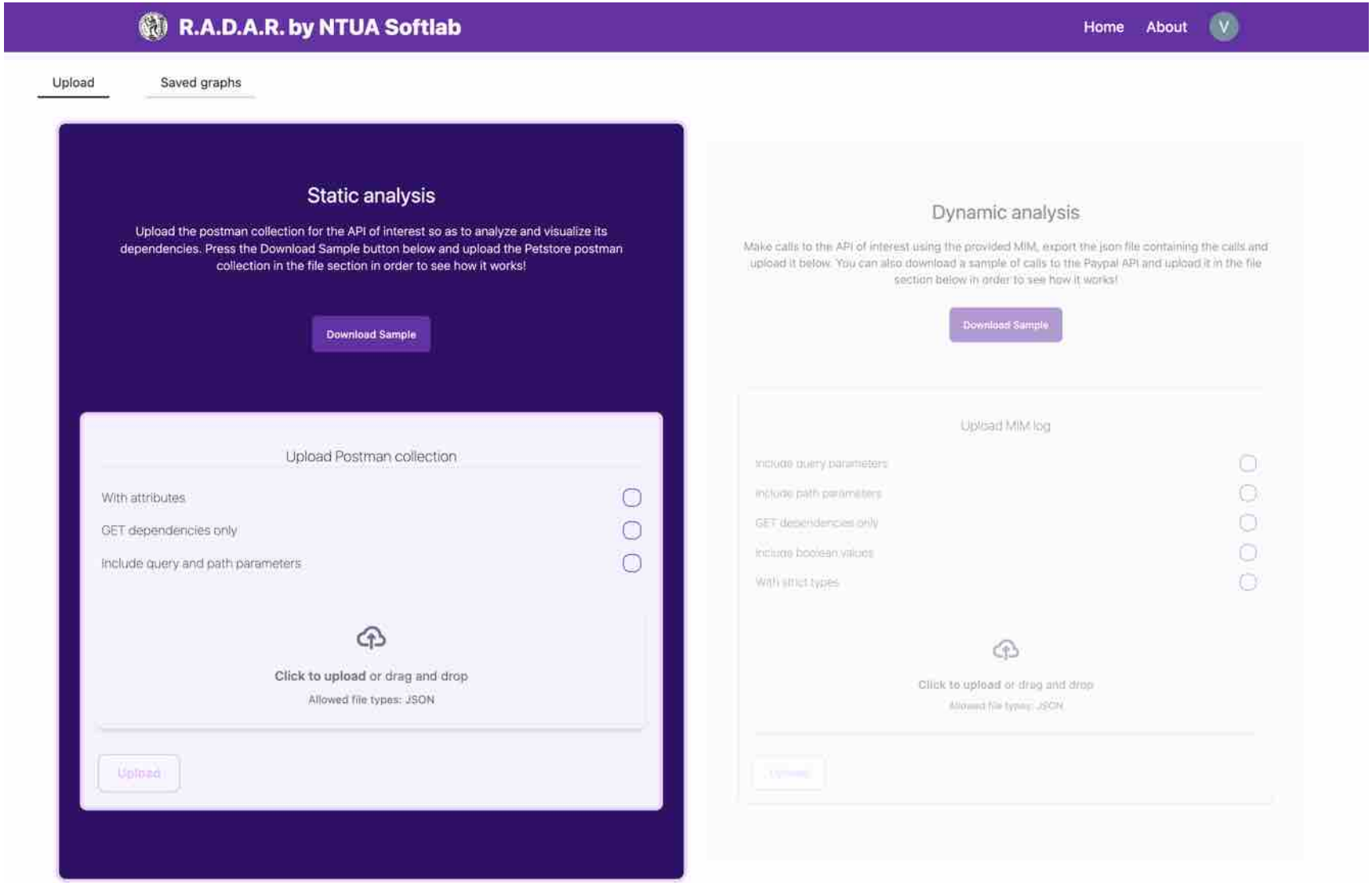
medium.com/@berrayana/selecting-the-right-open-source-api-gateway-technology-eb295f076939

APIs are(?) apps

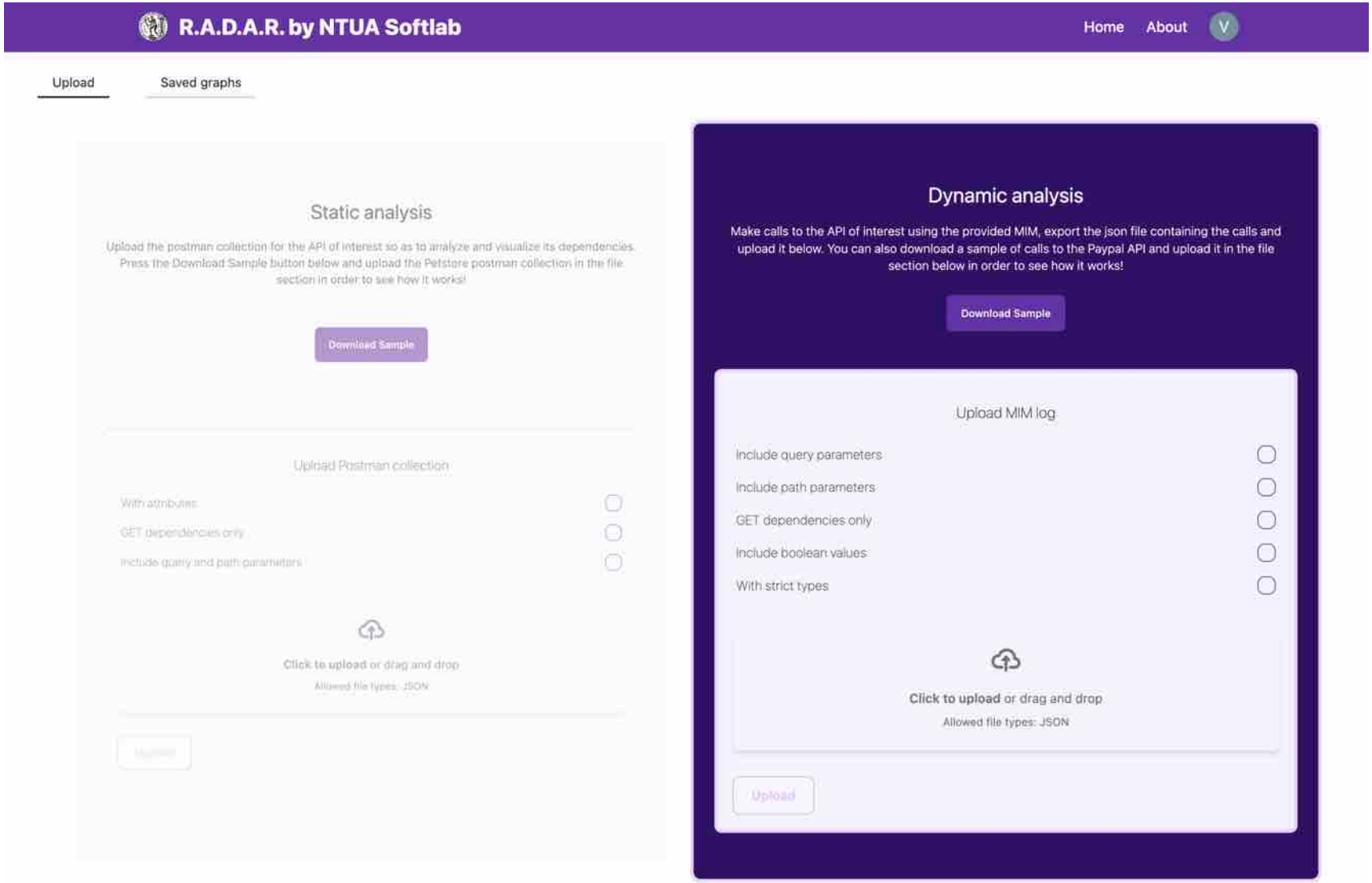




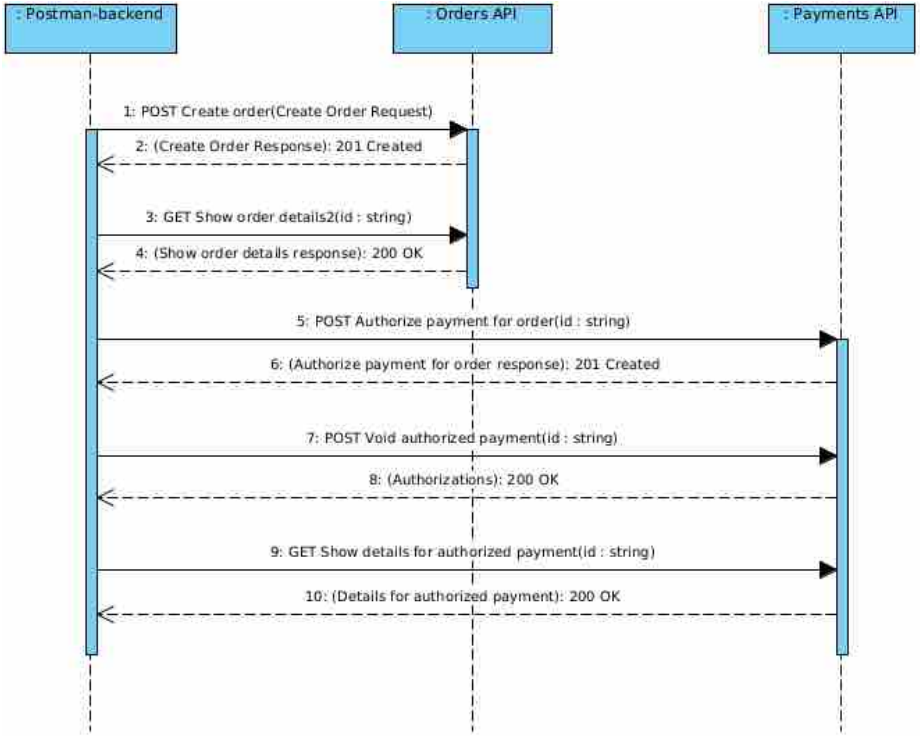
APIs++: radar.softlab.ntua.gr

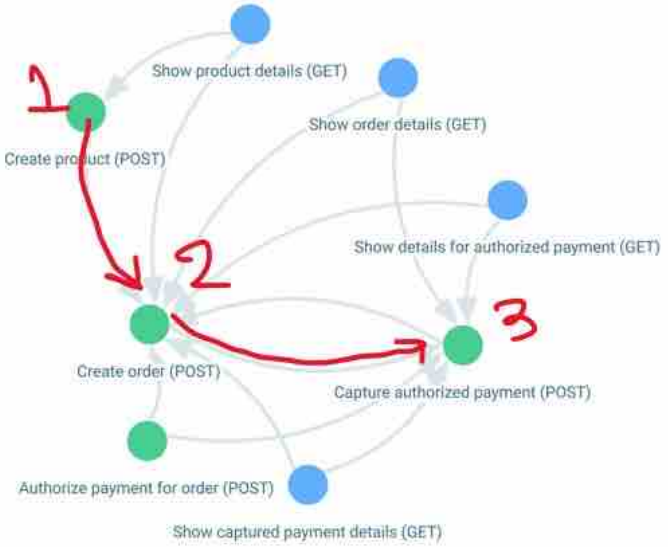


APIs++: radar.softlab.ntua.gr

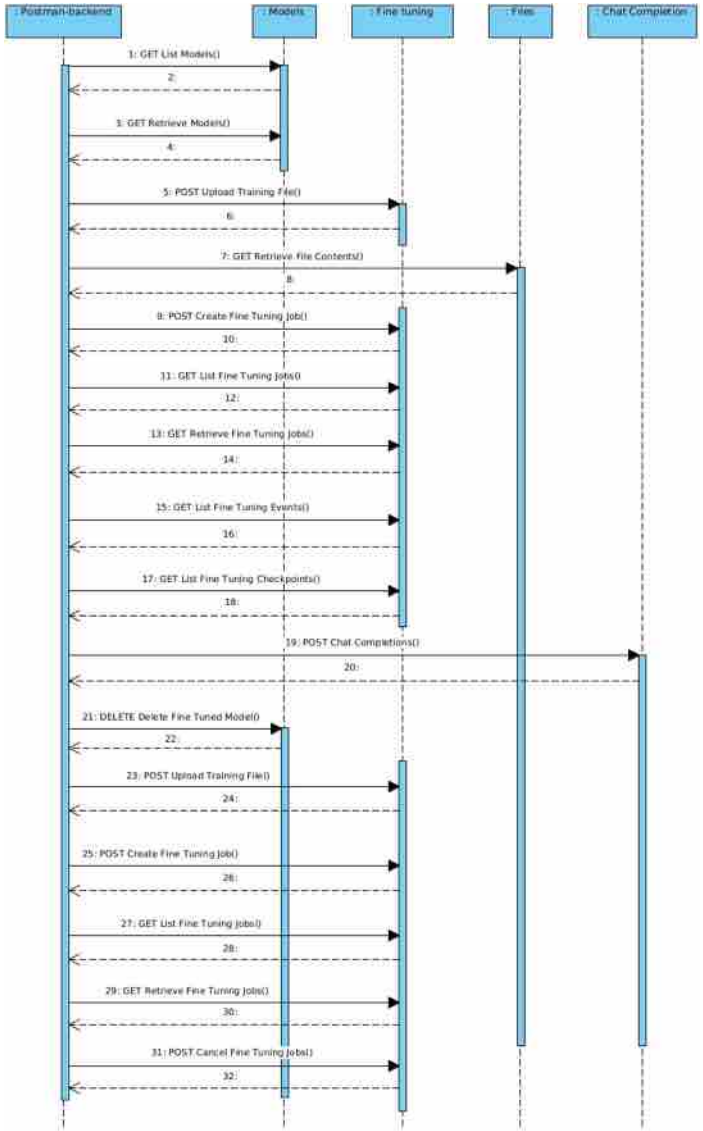


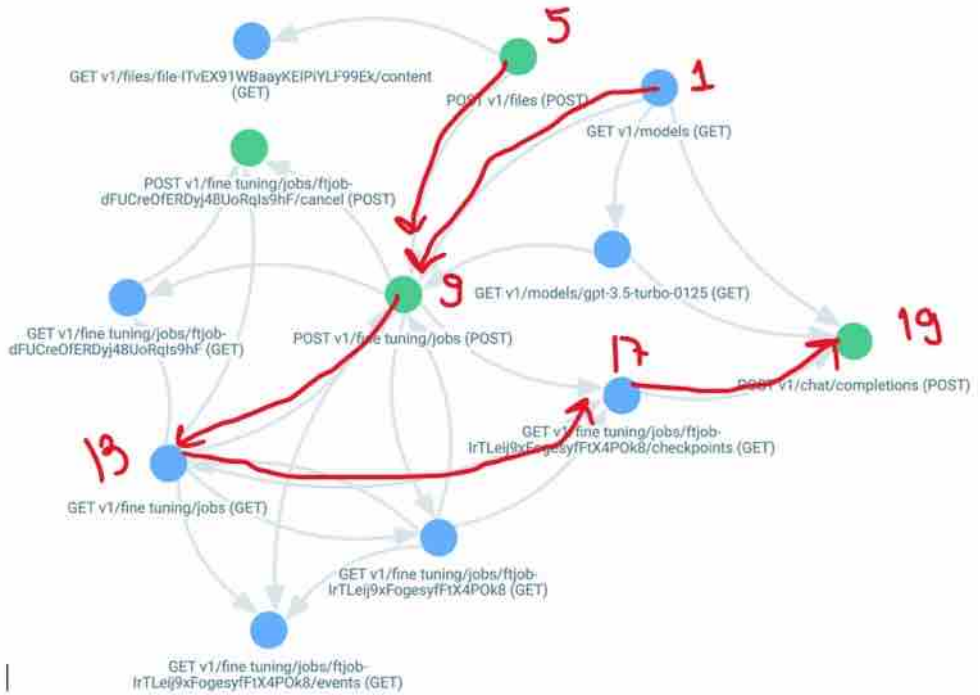
APIs++: radar.softlab.ntua.gr





APIs++: radar.softlab.ntua.gr





API semantics

OpenAPI

```yaml
paths:
  /orders/{orderId}:
    get:
      summary: Retrieve order details
      parameters:
        - name: orderId
          in: path
          required: true
          schema:
            type: string
        - name: expand
          in: query
          required: false
          schema:
            type: string
            enum: [items, payments]
      responses:
        '200':
          description: Order retrieved successfully
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/Order'
components:
  schemas:
    Order:
      type: object
      required:
        - id
        - amount
      properties:
        id:
          type: string
        amount:
          type: number
        currency:
          type: string
```

gRPC

```proto
syntax = "proto3";

package example.weather;

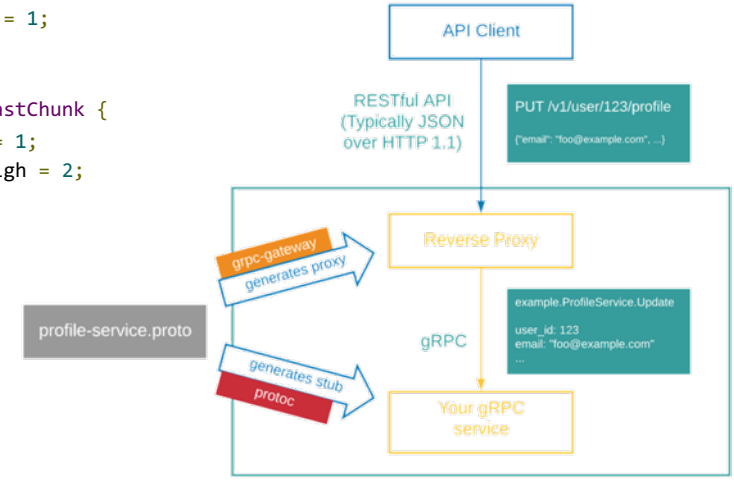
// Service definition: RPC methods with request/response types
service WeatherService {
  rpc GetCurrent (GetCurrentRequest) returns (GetCurrentResponse);
  rpc StreamForecast (ForecastRequest) returns (stream ForecastChunk);
}

// Message definitions: strongly-typed schemas
message GetCurrentRequest {
  string city = 1;
}

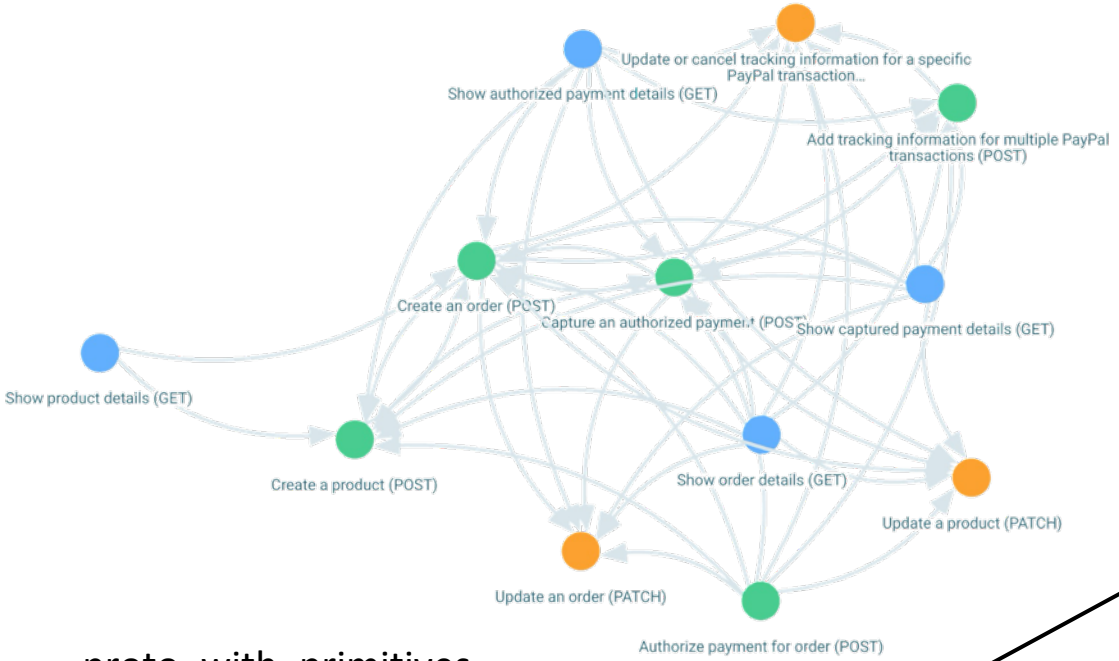
message GetCurrentResponse {
  float temperature = 1;
  string units = 2;
}

message ForecastRequest {
  string city = 1;
}

message ForecastChunk {
  string day = 1;
  float tempHigh = 2;
}
```



API semantics



proto_with_primitives

proto_without_primitives

